

金仓数据库适配：AI 辅助研发经验分享

项目：ThingsBoard 4.1 适配人大金仓 KingbaseES V8 (V008R006C009B0014) 周期：2026-08-27 ~ 2026-09-21
(日历跨度近 4 周，实际投入约 2 周 / 10 人日) 交付：分支 feature/kingbase-V008R006C009B0014-pg-oracle |
工具：Claude Code 面向：研发中心内部

一、先说结论

项目	结果
交付内容	一套代码同时支持 PostgreSQL 与金仓 pg / oracle 两种兼容模式
运行期 Java 代码 / SQL 查询 / 方言改动	0 处
新增内容	7 个文件：安装期自动探测逻辑 + 1 个 SQL 补丁函数 + 测试参数化 + 文档
对现有 PostgreSQL 的影响	实测为零（同库跑原始代码与新代码，失败清单完全一致）
验证规模	950 个 dao 用例 × 4 个环境 × 多轮，另有 9 个安装期单元测试
工作量	纯人工评估 32 人日 → 实际 10 人日，节省约 22 人日，效率约 3 倍

一句话：AI 把"做方案、跑验证"的成本降到很低，人把精力放在"该做哪个方案"上。

二、如果换组里另一个人来做

1. 要花多久

按"熟悉 PG、没接触过金仓的普通开发"评估：

阶段	人工	说明
安装金仓环境	2 天	查手册、装依赖、处理 license、调端口和权限
申请试用授权文件	1 天	走流程等授权
初始化三种兼容模式	2 天	每种模式要单独 initdb，模式只在首次初始化时生效
熟悉金仓使用	1 天	ksql、系统表、参数、兼容模式差异
摸底 + 适配开发	20 天	逐个排查报错、改代码、改方言
自测、修复	6 天	回归、修 bug
合计	32 天	

2. 会遇到什么问题

下面每一条，都是我这次真实踩过的：

可能遇到的问题	本次的真实情况	如果没发现会怎样
装好环境后不知道从哪下手	金仓差异点散落在函数、空串、大小写、运算符等各处，没有现成清单	只能一个个报错去查，20 天的开发量主要耗在这里
直接让 AI "去适配"，改动面失控	我要求"三种模式都支持"后，AI 改了 165 处查询 + Hibernate 方言层，功能跑通了，但风险很大	带着大改动上线，原有 PG 功能有没有被影响说不清
驱动和方言选错路	第一版用金仓自带驱动（jdbc:kingbase8），发现需要配套修改方言，工作量大；后来实测 PG 官方驱动 + 原 PG 方言就能支持 pg / oracle 两种模式	在方言改造上投入大量时间
版本没对齐到小版本	早期在金仓 C007 上验证"零改动能跑通"，换成客户的 C009 后同样代码 448 个用例失败，而且 C007 只有两种模式	把版本差异误判成"代码退化"，方向全错
把项目已有问题当成金仓问题	测试里有 10 个失败（Redis 镜像拉不到、自定义表没建、性能超时等），和金仓无关	白白为这 10 个"假问题"做适配
不报错的隐蔽问题	oracle 模式默认把空串当 NULL；mysql 模式 变成逻辑 OR、静默返回错误结果	测试可能是绿的，问题留到现场才暴露

三、我为什么快

环节	人工做法	我的做法
装环境、初始化模式	自己查手册、反复试	把测试机凭据交给 AI，它自己 SSH 登录安装、切模式、排障；一开始给的库连不通，它自己报告并换了另一台继续
熟悉金仓	先学再干	边干边学：AI 跑探针 SQL 实测行为，我看不懂就追问
摸底	一个个报错查	950 个用例 × 多环境全量跑，AI 自动比对失败清单
归因	凭经验判断	先用原始代码跑基线，只看"基线没有、现在有"的失败
开发	改代码 → 手工测几个场景	每次改动都跑 PostgreSQL / 金仓 pg / 金仓 oracle 多环境矩阵 全量回归
长时间测试	盯着等	单轮全量测试约 17 分钟，四环境串行一个多小时，全部丢后台，跑完再看结论；跨晚上的任务第二天早上直接看结果
方案试错	试错成本高，不敢试	mysql 模式完整做了一版（约 10 小时）再推翻，代价可接受
文档	最后补	基于实际代码让 AI 生成设计文档、部署文档和 PDF

说到底，快的原因有三个：

- 1. AI 直接上机干活：环境、测试、排障不用人中转每一步，原来 6 天的环境准备大幅压缩。
- 2. 用数据代替经验：基线对照加矩阵回归，2740 条报错归结到一个根因，开发量从"大改"变成"一个 SQL 函数"。

3. 试错成本低，敢先做再判断：三模式方案能低成本完整做出来再推翻，过去不敢这么干。

四、过程回顾：五个阶段

阶段	时间	做了什么	关键指令（原话）
1. 可行性评估	8/27 - 9/02	AI 登录测试机、装授权、跑探针 SQL 和性能对比，出了第一版实现（feature/kingbase-support，12 个提交）	"root/xxx 这个机器登录密码给你，你直接验证给我结果"；"我想适配这个金仓不影响现在原始的功能，确定能行吗？"
2. 文档化 + 追问到懂	9/14 - 9/17	让 AI 基于代码反向生成设计文档，看不懂的逐条追问，直到自己能讲清楚	"使用金仓的方言改动大对吗？"；"这个地方我没看懂什么意思"
3. 基线对照归因	9/18	用未修改的原始代码在金仓 pg 模式跑测试，失败清单与 PG 逐项一致，证明 pg 模式零适配可用	"直接用原来没有修改的任何代码跑，可以临时切换分支"
4. 扩到三模式，主动叫停	9/20	AI 实现了三模式（165 处查询 + 方言层），我评估风险后叫停，一期只做 pg + oracle	"现在三种模式改动代码太多了，会不会存在风险"；"如果不考虑 mysql 模式，代码是不是不用做任何改动？"
5. 零改动路线交付	9/20 - 9/21	从 master 重开干净分支，定位 oracle 模式唯一差异（concat 被遮蔽），补一个 SQL 函数；矩阵验证后交付，71 环境实跑一次通过	"安装期自动执行，能否在安装期检查 DBA 权限，没有权限就报错"；"只要和这次兼容有关的必要文件"

最终的 4 环境验证矩阵（以 PostgreSQL 15.4 为基线）：

环境	代码	Failures	Errors	与基线对比
PostgreSQL 15.4	原始代码	4	6	基线
金仓 pg 模式	未改动	4	6	逐项一致
金仓 oracle 模式	未改动、无补丁	128	320	+441
金仓 oracle 模式	未改动、有补丁	4	6	逐项一致

五、可复用的方法

1. 给真实环境，让 AI 自己验证，不要让它凭知识回答。金仓的很多行为从文档推不出来，只能实测。前提是只用自己可控的测试环境，生产凭据不要交出去。
2. 先定边界再动手。我的第一条要求一直是"可以开发，但不能影响原有功能"，所有方案都按这条来衡量。
3. 先建基线，再看差异。任何适配、迁移类工作，都先用原始代码在目标环境跑一轮。这是本次最省时间的一招。
4. 每次改动都跑多环境矩阵。人很难坚持每次全量回归，AI 可以，"零影响"的底气就来自这里。
5. 长任务丢后台。全量测试挂后台跑，人去做别的事，跑完再看结论。

- 6. 看不懂就追问，直到自己能讲清楚。判断标准：能不能脱离 AI，把这个技术决策给领导或同事讲一遍。
- 7. 主动质疑改动量，该砍就砍。AI 会忠实执行需求，不会主动建议"这个放到二期"。多问一句"改动多吗？有风险吗？去掉 X 是不是就不用改了？"
- 8. 方案定型后从主干重开干净分支。探索期分支里堆满了验证脚本和试错代码，不要直接交付。本次重开了两次，最终只剩 7 个文件。
- 9. 把踩坑结论沉淀下来，跨会话复用。本次跨十几个会话，靠 CLAUDE.md（项目结构、构建命令）和记忆文件（实测结论、版本号、环境状态）保持上下文。正因为记下了 C007 的版本号，才没把 C009 的 448 个失败误判成代码退化。

六、人必须把住的四道关

AI 能干活，但下面这四类问题它不会替你兜底。共同特点是**都不报错**，测试可能还是绿的。

关口	本次的真实案例	怎么把住
范围与风险	三模式方案改了 165 处查询 + 方言层，AI 没有建议砍掉 mysql 模式	人来判断改动量和收益是否匹配
静默失效的逻辑	AI 最初按 JDBC 产品名判断是不是金仓，但 PG 驱动连金仓时返回的是 "PostgreSQL"，导致金仓分支全部没生效，review 时才发现	测试全绿时多问一句"它到底走没走那条路径"
结论的时效性	C007 上验证通过的结论在 C009 上完全失效	换小版本必须重跑回归，不沿用旧结论
业务语义变更	mysql 模式强制大小写不敏感，设备名 CaseDev-A 和 casedev-a 会被判为重复	这是业务决策，要业务侧确认

七、可直接套用的提示词模板

评估阶段

帮我评估当前项目切换到 <目标环境> 是否可行。<环境地址与凭据>，你直接登录验证给我结果，不要只做静态分析。前提是不能影响现有 <原有环境> 的任何功能，不可用的功能单独整理给我。

归因阶段

先用未修改的原始代码在 <目标环境> 跑一轮完整测试作为基线，再用改动后的代码跑一轮，只把"基线没有、现在有"的失败列给我。后台跑，跑完告诉我。

决策阶段

这个方案改动多吗？会不会有风险？如果不考虑 <某个范围>，是不是代码就不用动了？给我两到三个方案的对比，说明各自代价，并说明你更推荐哪个。

基于最新的 master 新建分支 <分支名>，只把有用的代码合过去，验证脚本和中间产物不要。然后 review 本次改动：改了哪些文件、为什么改、对原有路径有什么影响。最后基于实际代码生成设计文档和部署手册，放到 docs 目录。

八、小结与建议

- **AI 承担的**：环境搭建与排障、全量回归的执行与比对、根因定位、代码实现、文档与 PDF 产出、跨一个月的上下文串联。
- **人承担的**：定范围、砍需求、质疑改动量、追问到懂、把控风险边界、拍板技术路线。
- **最有价值的一次干预**：在三模式方案已经"跑通"的情况下，问了一句"只考虑 pg 和 oracle，是不是代码不用做任何改动？"。这一问把交付从"改 165 处查询 + 方言层"变成了"零 Java 改动 + 一个 SQL 函数"。

AI 让"把方案做出来"的成本大幅下降，"判断该做哪个方案"的价值反而被放大了。

后续建议：

1. 这套方法（真实环境验证 → 基线对照 → 矩阵回归 → 人做取舍 → 干净分支交付）可以直接用于其他国产数据库和中间件的适配。
2. 第七节的提示词模板可以作为团队标准用法推广。
3. 金仓 mysql 兼容模式如果后续确有需求，二期单独立项评估，需要业务侧先确认大小写不敏感是否可以接受。

附：相关产出物

产出	位置
交付代码	分支 <code>feature/kingbase-V008R006C009B0014-pg-oracle</code>
部署与技术说明	<code>docs/kingbase-pg-oracle.md</code>
oracle 补丁脚本	<code>dao/src/main/resources/sql/schema-entities-kingbase-oracle-addon.sql</code>
安装期逻辑	<code>application/.../install/SqlEntityDatabaseSchemaService.java</code>
一键部署	<code>docker/kingbase/docker-compose.yml</code>